Bridging Compute and Memory Fabric: Data Services for Direct XPU Access

Gaurav Agarwal

Senior Distinguished Engineer, Marvell



OCP FUTURE TECHNOLOGIES SYMPOSIUM

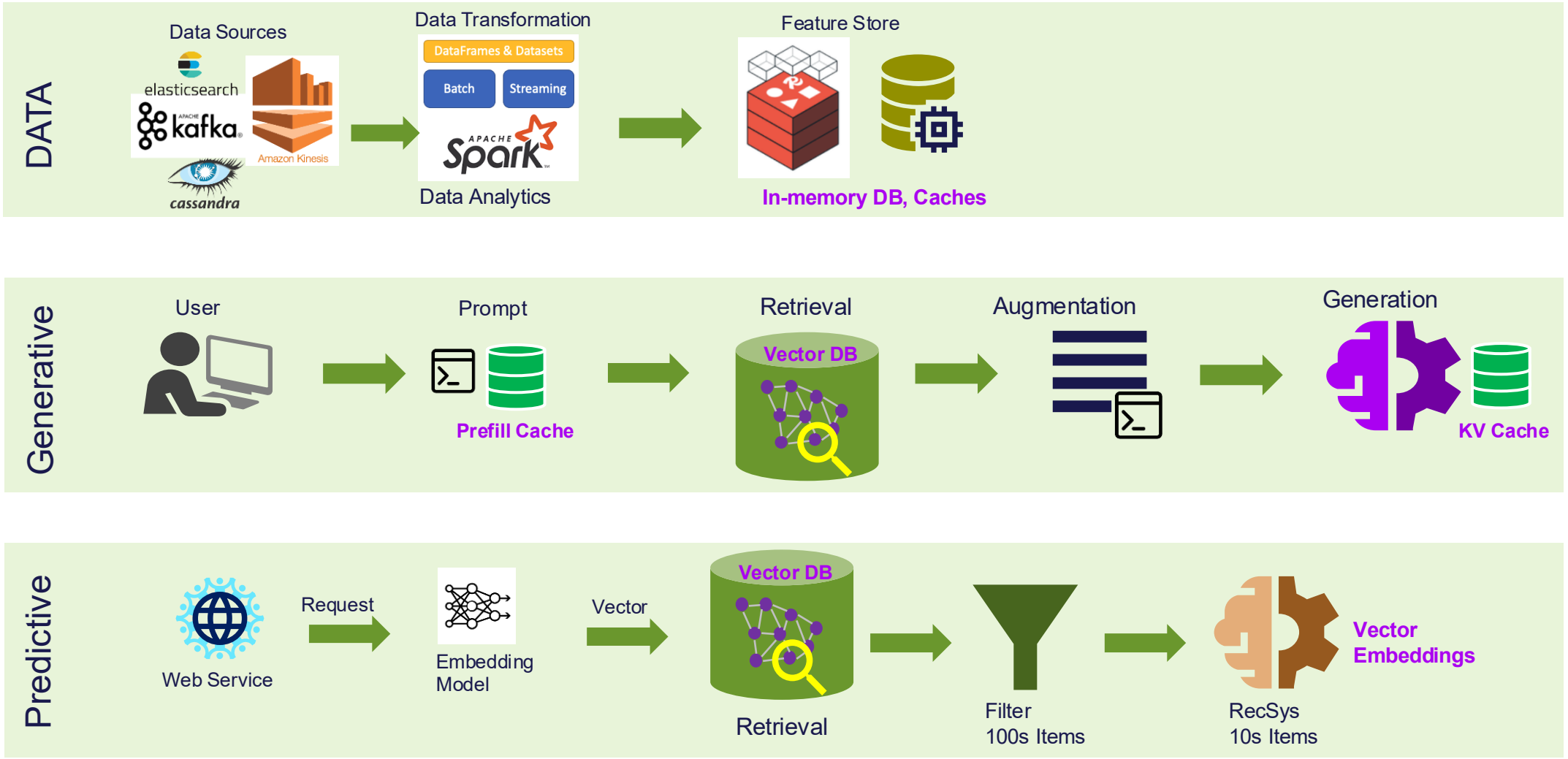
October 14, 2026
San Jose, California

Forward-looking statements

Except for statements of historical fact, this presentation contains forward-looking statements (within the meaning of the federal securities laws) including statements related to future revenue, future earnings, and the success of our product releases that involve risks and uncertainties. Words such as “anticipates,” “expects,” “intends,” “plans,” “projects,” “believes,” “seeks,” “estimates,” “can,” “may,” “will,” “would” and similar expressions identify such forward-looking statements. These statements are not guarantees of results and should not be considered as an indication of future activity or future performance. Actual events or results may differ materially from those described in this presentation due to a number of risks and uncertainties.

Forward-looking statements are only predictions and are subject to risks, uncertainties and assumptions that are difficult to predict, including those described in the “Risk Factors” section of our Annual Reports on Form 10-K, Quarterly Reports on Form 10-Q and other documents filed by us from time to time with the SEC. Forward-looking statements speak only as of the date they are made. You are cautioned not to put undue reliance on forward-looking statements, and no person assumes any obligation to update or revise any such forward-looking statements, whether as a result of new information, future events or otherwise.

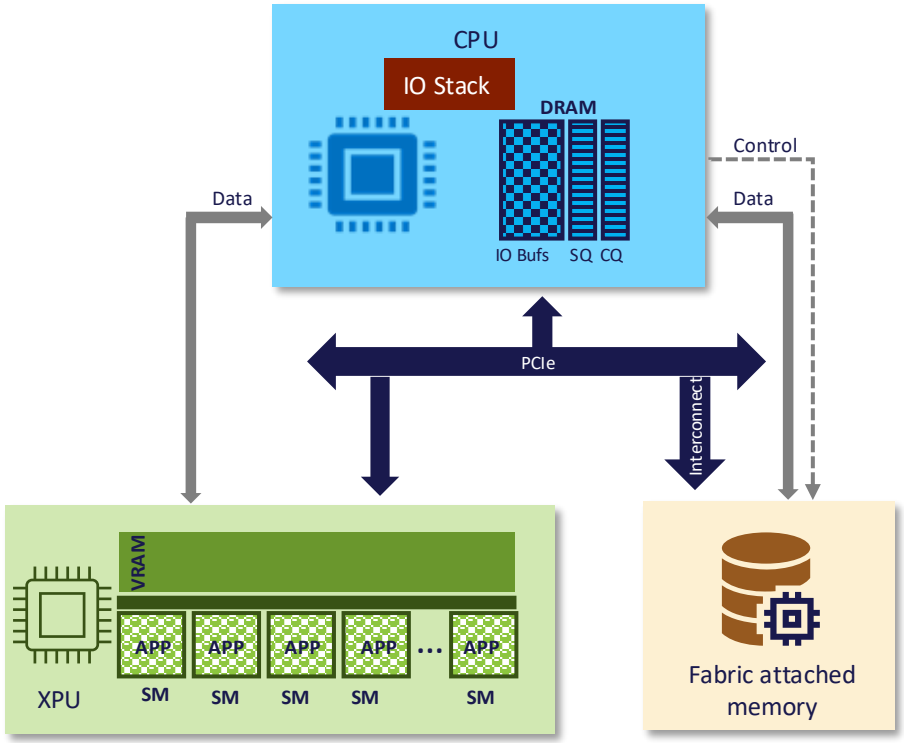
Data-centric AI pipelines



Data-centric workloads: key elements

- High-performance data movement
- Universal data access methods
- Open protocols
- Efficient data storage
- Scalable, power-efficient, cost-effective, extensible solutions
- …

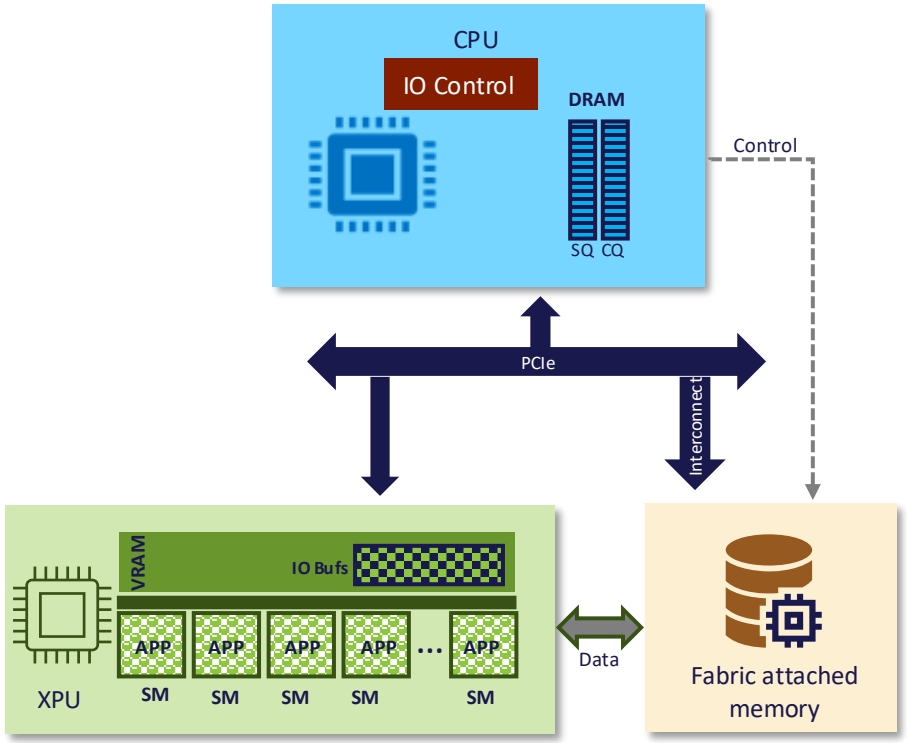
CPU driven I/O



- CPU
  - Transfers data to/from far memory
  - Prepares and plumbs the data into XPU
  - Orchestration – kernel launches
- Suitable for large data objects known to CPU
- Issues
  - Significant data movement overhead
  - Per operation latency O(us)
  - XPU<->CPU synchronization
  - Limited concurrency in CPU

Misfit for accelerator-centric architecture!

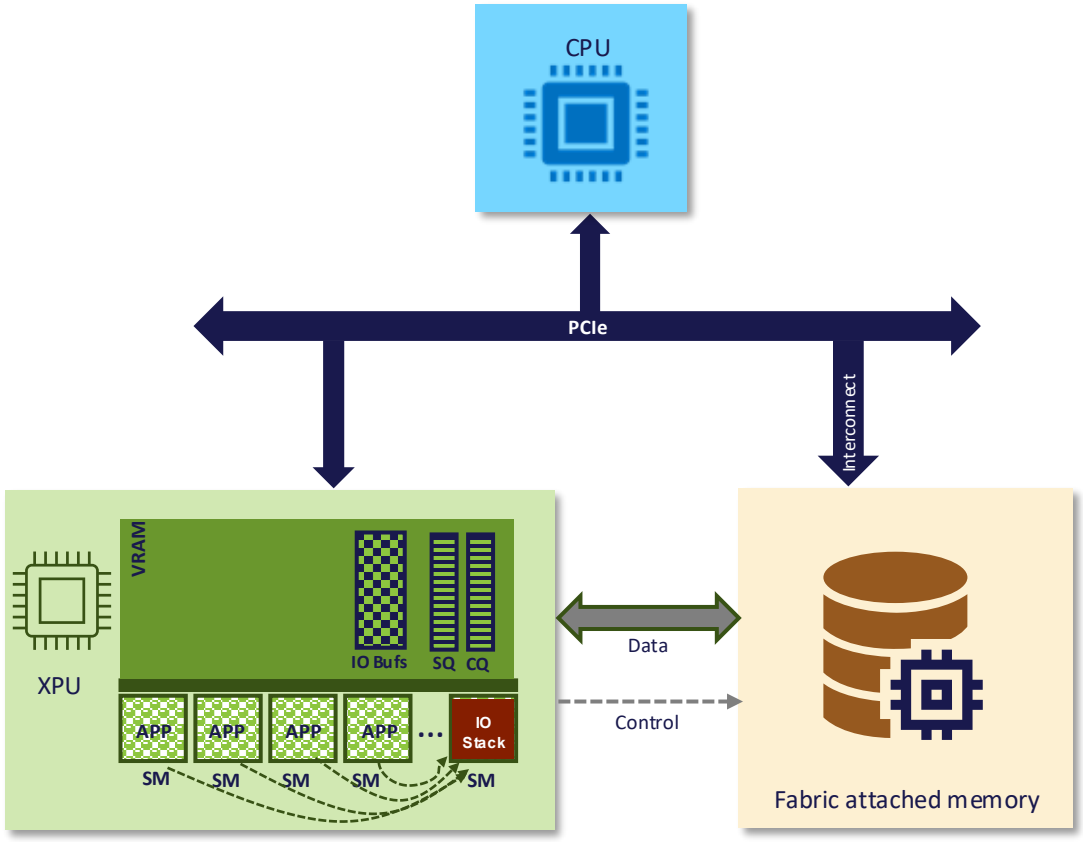
CPU assisted XPU direct data transfer



- CPU
  - IO control (IO initiation, completions, etc.)
  - Orchestration – kernel launches
- XPU
  - Data transfer (DMA, SM kernel, GDS, etc.)
- Issues
  - Per operation latency O(us)
  - XPU<->CPU synchronization
  - Limited concurrency in CPU

APP controlled fine-grained I/Os overwhelm CPU!

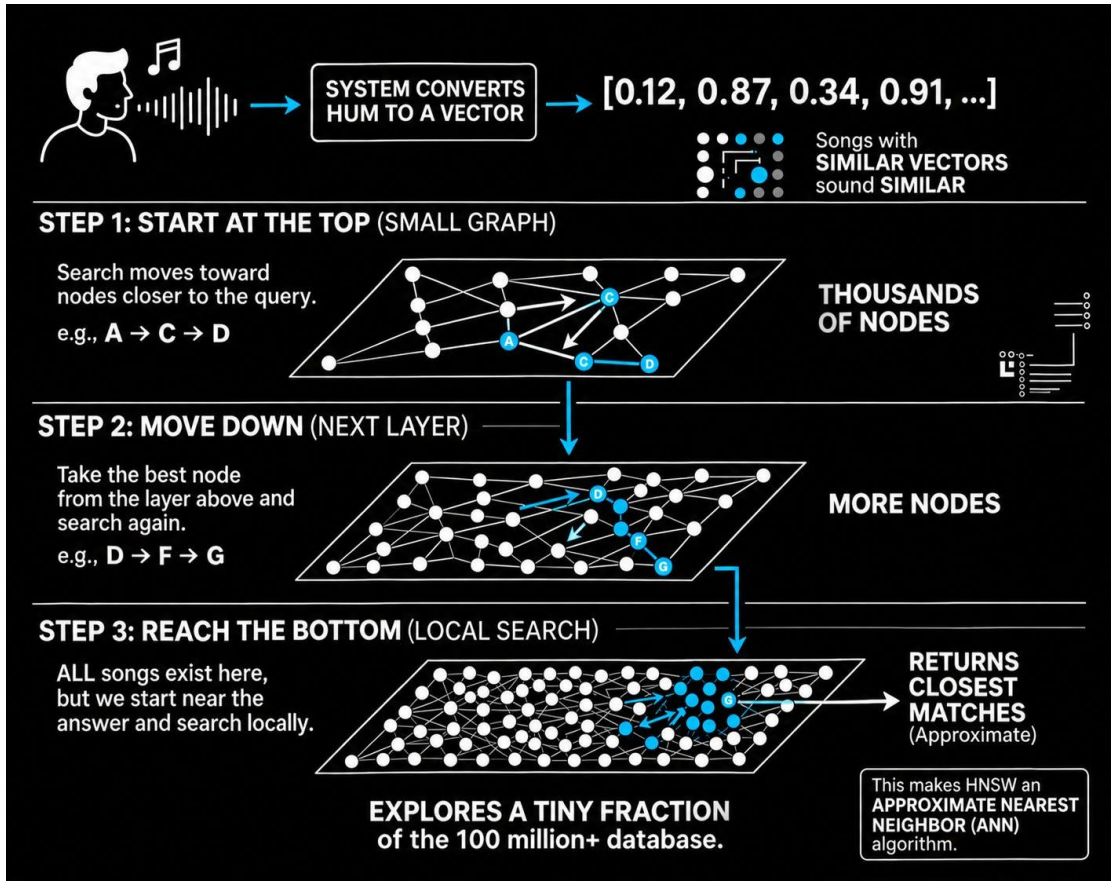
XPU driven data transfer



- CPU
  - Orchestration – kernel launches
- XPU
  - Full IO stack (control and data transfer)
- Issues and challenges
  - I/O threads synchronization
  - Polling for completions
  - Protections and access control
  - Stalls – due to I/O latency
  - I/O amplification (raw data access)

I/O amplification remains a serious issue!

HNSW graph index navigation



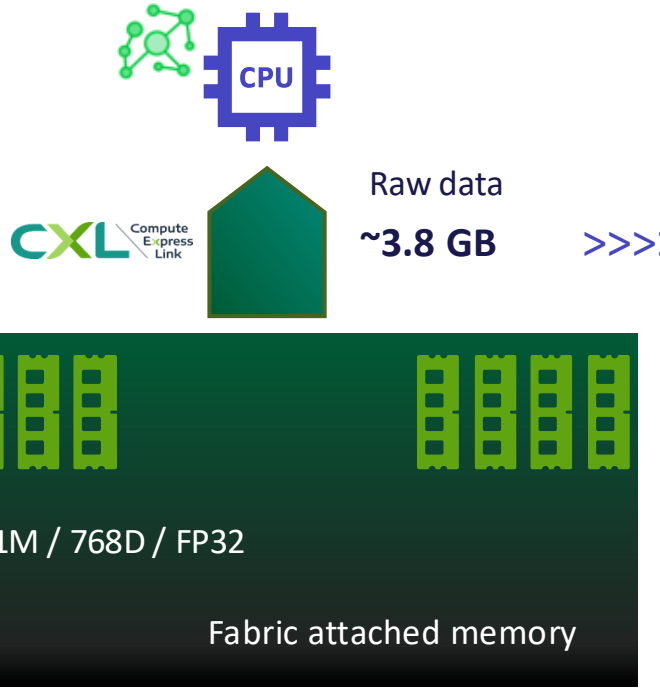
Source: https://newsletter.systemdesign.one/p/hnsw-vector-search-explained

- Graph nodes are small O(100B)
- Prefetching is not an option
- I/O latency dominates performance
- I/O amplification factors
  - Transfer protocol overhead
  - Media I/O block size

Vector search data reduction

Compute centric

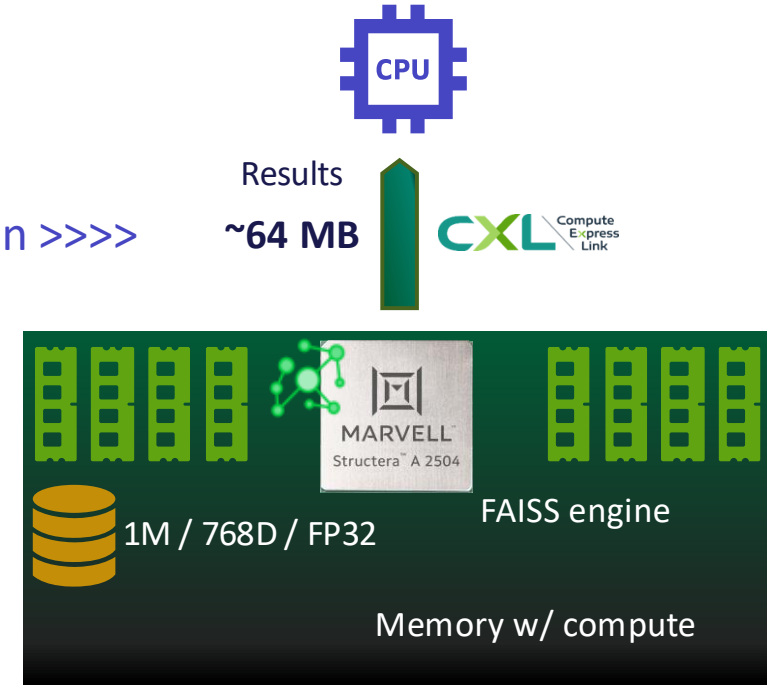
Raw data is pulled across fabric
Query processing on host



>>>> Reduction >>>>

Data centric

Offload query
Pull results



Scenario

Number of vectors: 1,000,000

Single vector size: 384B
Dimensions: 96
Precision: FP32

Query param ef_search: 100
Query batch size: 16K
Top K: 10

No. of comparisons: ~600
= ef_search * log(N)

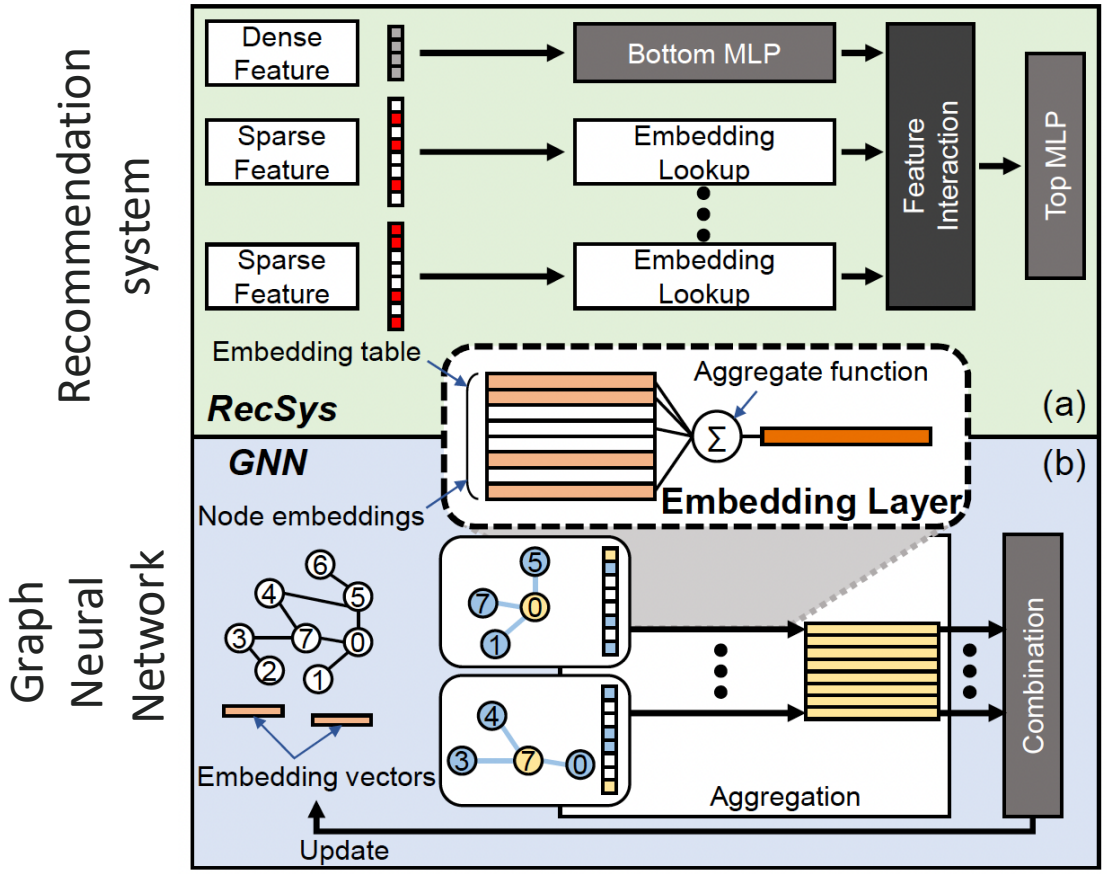
Query Batch memory reads: ~3.8GB
= 600 * 384B

16K batch query result size: ~64MB
= 16K * 10 * 384B + 16K * 10 * 4B

Source: Marvell (OCP Global Summit 2025)

Near memory compute reduces data movement by 60X

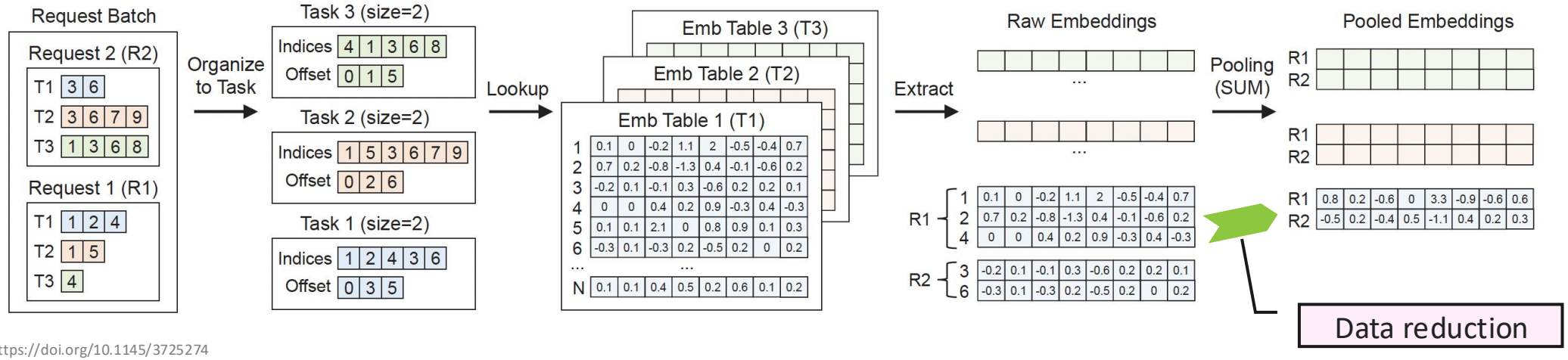
Vector embeddings



Source: CLAY https://doi.org/10.1145/3650200.3656595

- Many AI systems use vector embeddings
  - Recommendation systems, knowledge graph reasoning, GNNs…
- Sparse to dense transformation
  - E.g., categorical data (liked videos, watched movies, etc.)
  - Aggregation function is low compute intensity
- Perfect candidate for disaggregated memory and near memory compute

Embeddings lookup and aggregation example

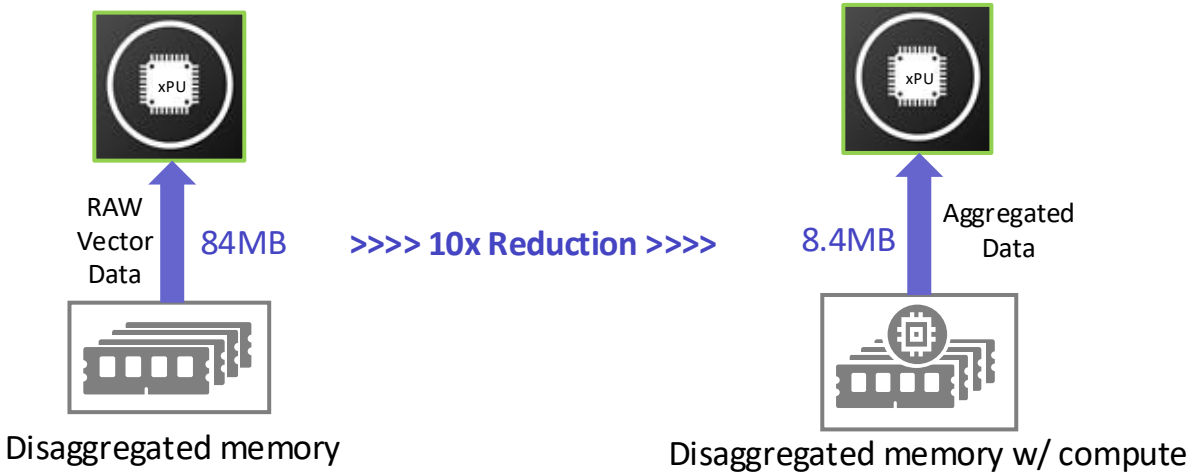


Source: CARINA https://doi.org/10.1145/3725274

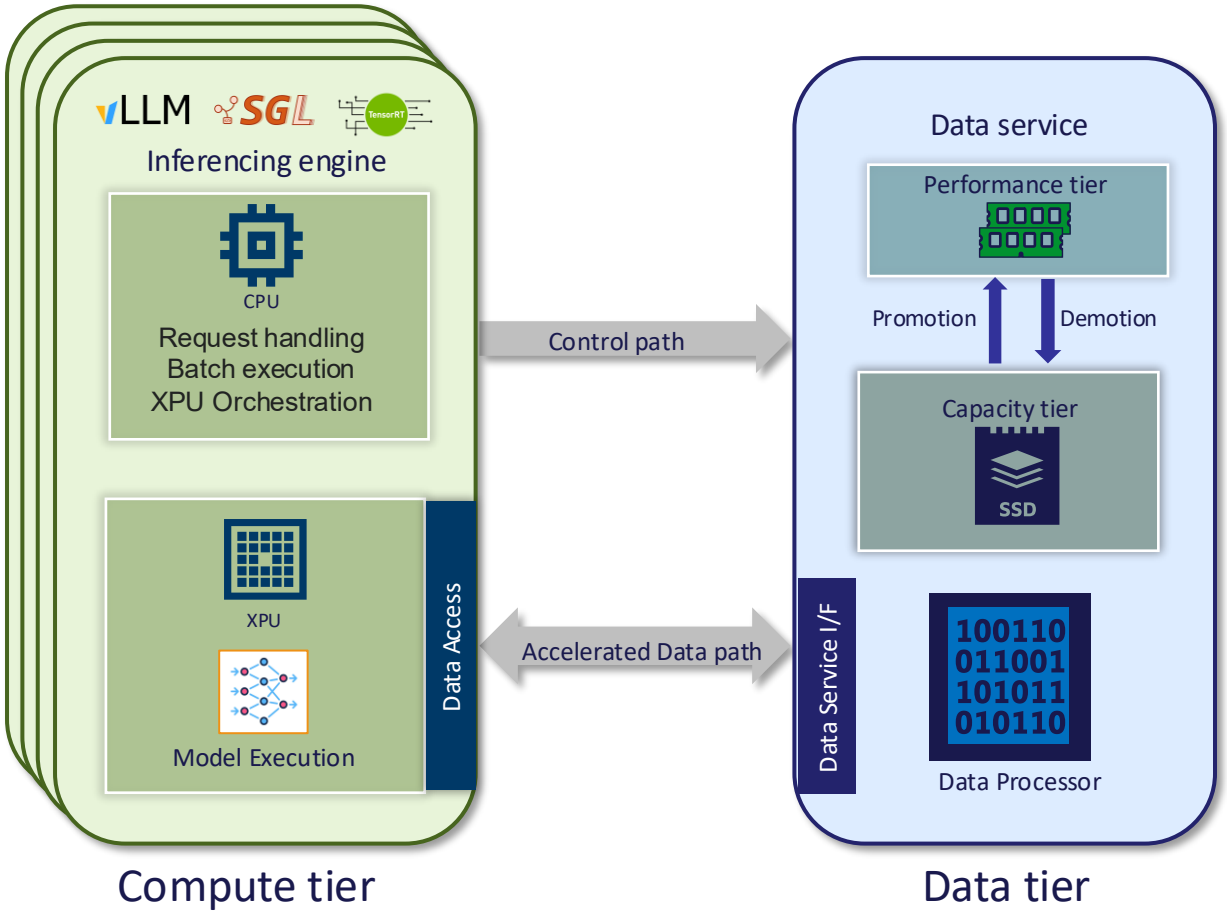
Scenario

B	Batch size: 2K requests
T	# Embedding Tables: 8
L	Lookups per sample per table: 10
D	Embedding Dimension: 128* floats

* 4 bytes per float (FP32)



Consider data services model for data-centric AI



- Efficient data storage
  - Multi-tier, smart placement, promotion and demotion
- Accelerated data transfer
- Data management
  - Transformations: storage and serving time
  - Security
- Unified API for data access
- …

© 2026 Marvell. All rights reserved.

Summary

1. **AI serving at scale is increasingly data-centric**, making memory capacity, data movement, and data access efficiency critical
2. **CPU-mediated I/O is a bottleneck** in accelerator-first architectures
3. **Near-Data Processing (NDP) minimizes data movement**, reducing I/O overhead and fabric bandwidth demands
4. **Unified APIs enable portable, AI-native data services** across composable memory fabrics
5. **Disaggregated memory enables independent scaling of compute and data** capacity to match workload demands, unlock more scalable, cost-effective, and energy-efficient AI infrastructure

OCP FUTURE TECHNOLOGIES SYMPOSIUM

October 14, 2026
San Jose, California

